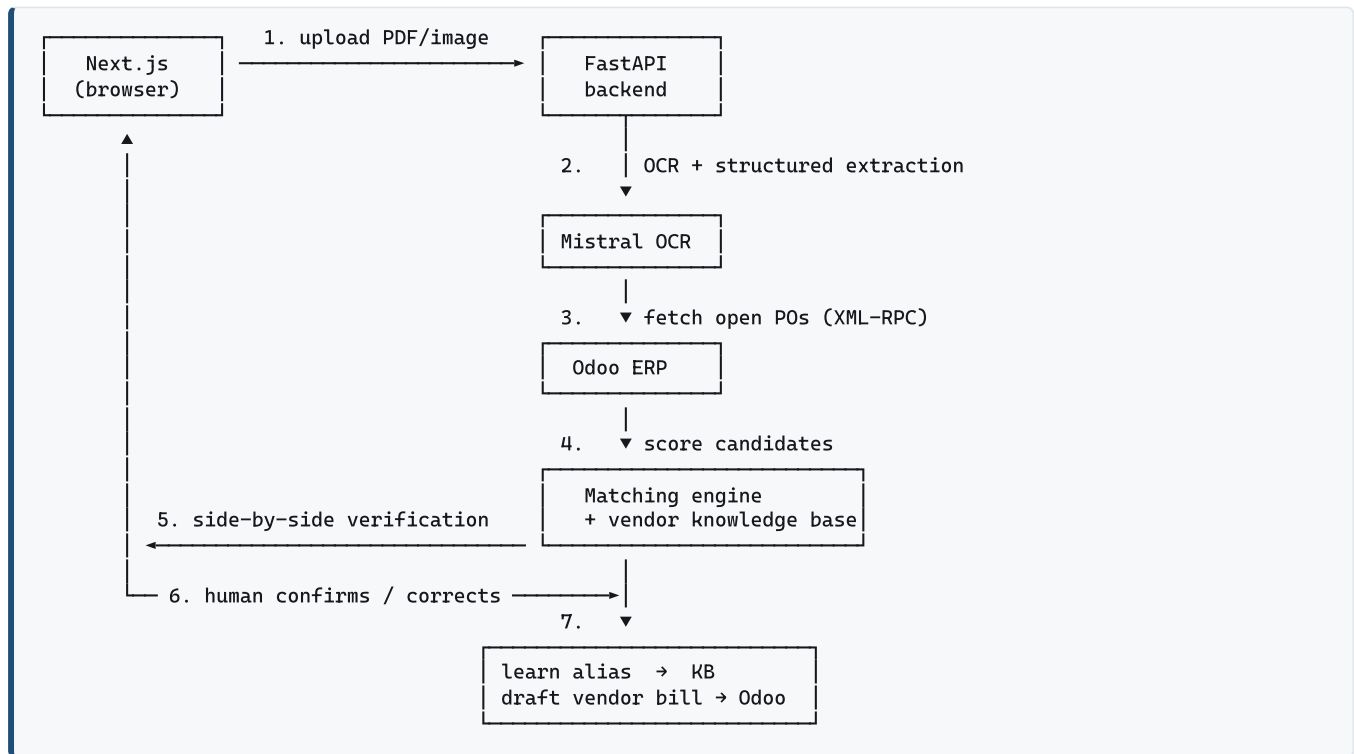


Overview & Decisions

What this system does

An Accounts Payable clerk receives a supplier invoice as a PDF or a phone photo. Today they open Odoo, hunt for the matching Purchase Order, eyeball the totals, and key in a vendor bill. This system does the hunting and the eyeballing, and asks the human only to approve.



The loop closes at step 7: every human correction teaches the knowledge base, so the same vendor matches automatically next month. That feedback loop — not the OCR — is what makes this worth building.

The seven steps in detail

#	Step	Owner	Notes
1	Upload invoice	Next.js → FastAPI	Multipart; MIME validated by magic bytes, not extension
2	Extract structured data	<code>ocr_service.py</code>	One Mistral call does OCR and JSON extraction
3	Fetch candidate POs	<code>odoo_service.py</code>	<code>purchase.order</code> in open states, lines batched in one read
4	Score candidates	<code>matching_engine.py</code>	Weighted 0–100, KB consulted first
5	Verify side-by-side	Next.js	PDF left, comparison table + score right
6	Confirm or correct	Next.js → FastAPI	User may pick a different PO entirely
7	Learn + push	<code>kb_service.py</code> , <code>odoo_service.py</code>	Alias upserted; draft <code>account.move</code> created

Locked decisions

These were settled before writing and are load-bearing — changing one changes several documents.

Decision	Choice	Why
Python runtime	3.12	The existing <code>server/venv</code> is 3.15.0. No cp315 wheels exist for <code>asyncpg</code> , <code>pydantic-core</code> , <code>greenlet</code> or <code>rapidfuzz</code> ; pip falls back to source builds and fails without MSVC Build Tools. <code>py -3.12</code> is already installed.
Tenancy	<code>organizations</code> table from day one	A SaaS AP tool is bought by a company. Clerk, controller and approver share one invoice queue and one vendor KB. Retrofitting an org table later means migrating every FK and every query. Cost now: one table, one FK.
Auth	Self-hosted JWT, argon2id + PyJWT	No third-party identity dependency. <code>passlib</code> is unmaintained and breaks on new Python releases; <code>python-jose</code> has a CVE history.
Odoo credentials	Per-organization, Fernet-encrypted	Each tenant has their own Odoo instance. The <code>ODOO_*</code> env vars remain a dev-only fallback.
Odoo write-back	Draft vendor bill via <code>action_create_invoice</code>	Odoo computes taxes, fiscal position, currency and the <code>purchase_line_id</code> back-links. Never auto-posts, so a bad match cannot become a live journal entry.
Processing model	Inline in the upload request	Mistral OCR on a 1–3 page invoice is 3–12s, which fits an HTTP request. <code>app/workers/</code> is scaffolded so moving to a queue is a small change, not a rewrite.
Odoo transport	<code>xmlrpc.client</code> inside <code>anyio.to_thread.run_sync</code>	xmlrpc is fully blocking. Calling it on the event loop stalls every concurrent request.
Money	<code>Decimal</code> + <code>NUMERIC(18,4)</code> end to end	Never float. <code>asyncpg</code> round-trips <code>Decimal</code> natively.
Database	PostgreSQL 18 (already running locally)	<code>gen_random_uuid()</code> is built in; JSONB for raw OCR payloads.
Frontend	Next.js 16.3, App Router	Current stable. Renames <code>middleware.ts</code> → <code>proxy.ts</code> and pins TypeScript to 6.x.
Frontend auth transport	httpOnly cookie + same-origin rewrite	Enables real server-side route protection and makes the PDF viewer work without an authenticated blob fetch. See §5.

Environment prerequisites

Verified on the target machine:

Tool	Status
Python 3.12	Installed (<code>py -3.12</code>) alongside 3.13 and 3.15
Node.js	v24.15.0
npm	11.12.1
PostgreSQL	18, service running
git	2.47.1 — but <code>c:\ocr</code> is not yet a repository
Docker	Not installed
Chrome / Edge	Both present (used to render these PDFs)

You will also need, from outside the machine:

- A **Mistral API key** (<https://console.mistral.ai>).
- An **Odoo instance** with an integration user. That user needs *Purchase* → *User* and *Invoicing* → *Billing* access rights. Generate the API key under *Preferences* → *Account Security* → *New API Key*. An odoo.sh trial is fine for development.

Build order

Follow this sequence. Each step is verifiable on its own, and the ordering front-loads the integrations with the most unknowns.

1. **Foundation** — recreate the venv on 3.12, install dependencies, create the database.
2. **Skeleton** — `config` → `errors` → `logging` → `middleware` → `db` → `main`. Verify `/health` and `/health/ready`.
3. **Schema** — models, mixins, Alembic init, first revision, `upgrade head`.
4. **Auth** — security, crypto, repositories, `auth_service`, `deps`, auth router. Register → login → `/auth/me` must work end to end.
5. **Odoo** — `odoo_service.py` and the `/odoo/*` endpoints. **Prove this against a real Odoo before going further**; it is the highest-uncertainty integration in the system.
6. **OCR** — `storage_service`, `ocr_service`, `POST /invoices/upload` with `auto_match=false`. Prove extraction on five real invoices and keep the JSON as fixtures.
7. **Matching** — `matching_engine.py`, unit tests first, against the fixtures from steps 5 and 6. This is the only component you can iterate on with no network access.
8. **The loop** — `kb_service`, `invoice_service` orchestration, `/confirm`, Odoo push.
9. **Frontend** — scaffold, types and lib primitives, auth slice, dashboard, upload, then the verification screen (build `InvoicePreview` first; it carries the most technical risk).
10. **Polish** — knowledge base admin, stats, integration tests, accessibility pass.

Version traps

Each of these was verified against a live registry, not recalled. Every one of them produces a confusing failure if ignored.

1. `pdfjs-dist` must be pinned to exactly `5.4.296`. `react-pdf@10.4.1` depends on that exact version. Declaring `^6` hoists 6.x to the top level while react-pdf keeps a nested 5.x, and the worker URL then resolves to the 6.x worker against the 5.x API: `The API version "5.4.296" does not match the Worker version "6.2.108"`.
2. TypeScript pins to `6.0.3`, not the `latest` `7.0.2`. `typescript-eslint@8.67.0` declares `typescript: ">=4.8.4 <6.1.0"`. TS 7 (the Go port) has no stable programmatic API yet, so linting cannot consume it.
3. `mistralai` v2 moved its imports. Use `from mistralai.client import Mistral`. The v1 path `from mistralai import Mistral` no longer resolves, and most tutorials still show it.
4. Next 16 removed `next lint` and the `eslint` key in `next.config`. `next build` no longer lints; ESLint runs as its own script with flat config.
5. Do not pin `starlette`. It has gone 1.x and FastAPI already constrains a compatible range; pinning it yourself causes a resolver conflict on the next FastAPI bump.

Out of scope for v1

Called out so nobody assumes otherwise:

- No CI pipeline, Dockerfile or deployment target. The runbook covers local development.
- `c:\ocr` is not a git repository. Initialise it, and add `.gitattributes` with `* text=auto eol=lf` before the first commit or Alembic revisions will churn CRLF.
- No background job queue. Uploads process inline.
- No email, billing, or password-reset flows.
- File storage is the local filesystem behind a `storage_service` interface, so swapping in S3 later touches one module.